Tehnologii Web

front-end ***back-end***

interacțiune Web

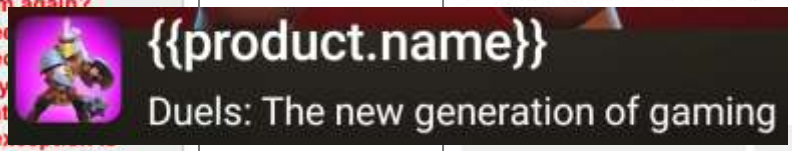
suita **Ajax** + aplicații ***mash-up*** + **JAMstack**

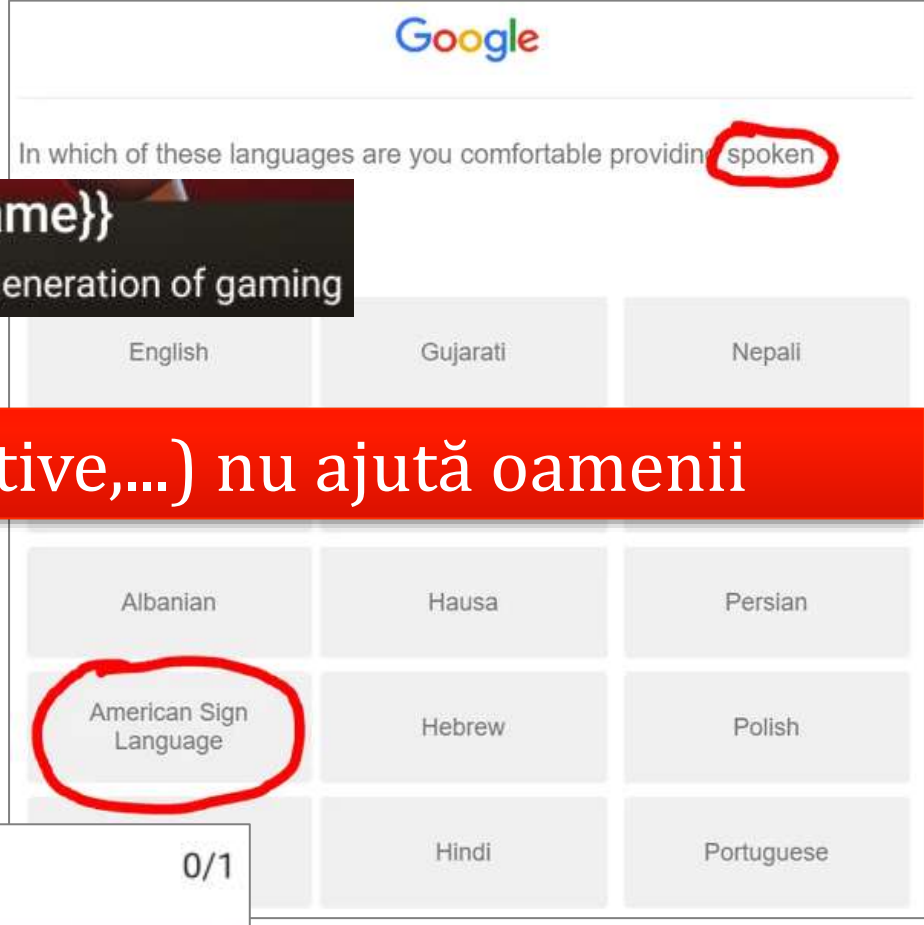
Dr. Sabin Corneliu Buraga – **profs.info.uaic.ro/~busaco/**

„Modul în care dăm
face mai mult decât ceea ce dăm.”

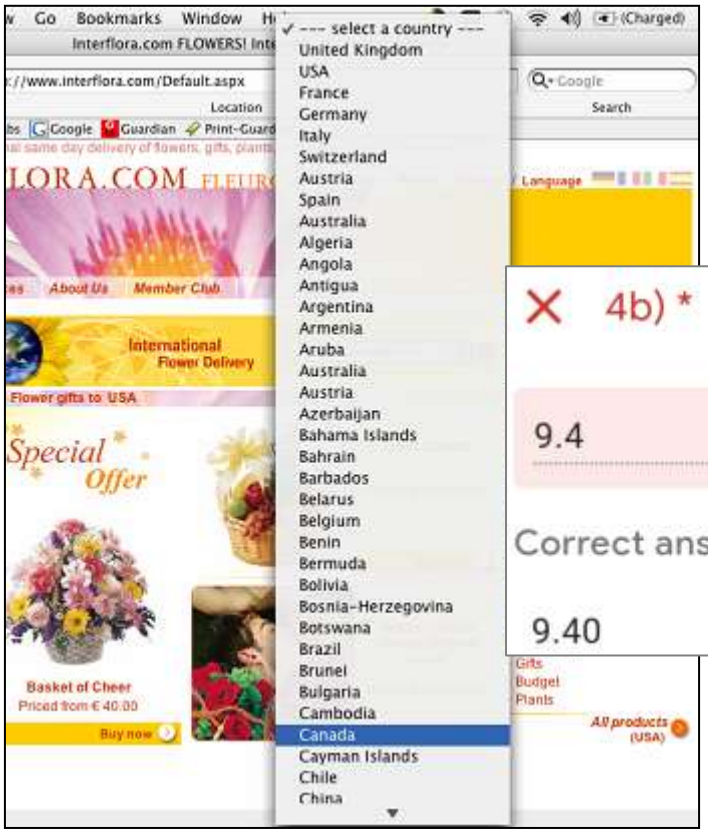
Pierre Corneille

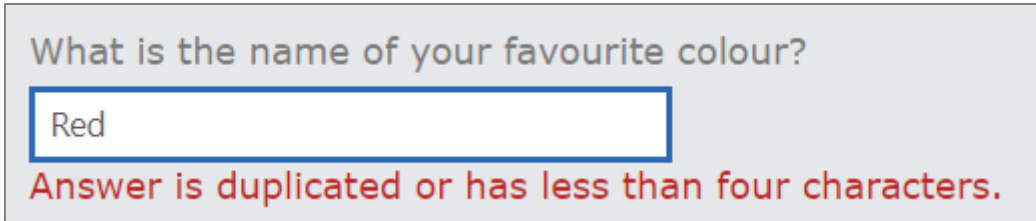
Cum facilităm interacțiunea
dintre utilizatorii (umani) și aplicațiile Web?





uzual, aplicațiile (Web, native,...) nu ajută oamenii





interacțiune web

Context mai larg:

HCI – *Human-Computer Interaction*

human – a person who tries to accomplish a goal

everyone who uses a (software) tool is driven by a motivation

computer – runs applications (software) locally and/or remotely

interaction – "dialogue" between humans and computers/things

interacțiune web

Interacțiunea dintre utilizator(i) și aplicații Web are loc prin intermediul unei interfețe Web

detalii la cursul opțional din anul III:
„Interacțiune om-calculator”
profs.info.uaic.ro/~busaco/teach/courses/hci/

Care e modalitatea de a transfera asincron date între client(i) și server(e) Web?

interacțiune web: ajax

Asynchronous JavaScript And XML
(Jeese James Garrett)

permite transferul asincron de date
între un document HTML redat de client (*browser*)
și o aplicație rulând pe un server Web

interacțiune web: ajax

O suită de tehnologii deschise:

limbaje standardizate de structurare – uzual, HTML – și de prezentare a datelor: CSS

interacțiune web: ajax

O suită de tehnologii deschise:

redare + interacțiune la nivel de client (navigator) Web via standardul DOM

interacțiune web: ajax

O suită de tehnologii deschise:

interschimb și manipulare de date reprezentate prin:

JSON

diverse dialecte XML (*e.g.*, RSS, KML,…)

HTML

alte formate – *e.g.*, CSV

interacțiune web: ajax

O suită de tehnologii deschise:

procesare folosind limbajul ECMAScript (JavaScript)

interacțiune web: ajax

O suită de tehnologii deschise:

transfer (a)sincron de date via HTTP
facilitat de obiectul **XMLHttpRequest**
disponibil via JavaScript la nivelul navigatorului Web

HTML Living Standard, 26 martie 2021
xhr.spec.whatwg.org

interacțiune web: ajax

Componenta de bază: obiectul **XMLHttpRequest**

permite realizarea de cereri HTTP – *e.g.*, GET, POST,… – dintr-un program rulând la nivel de client (*browser*) spre o aplicație / un serviciu Web existent(ă) pe server, în mod **asincron** ori **sincron**

interacțiune web: ajax

Componenta de bază: obiectul **XMLHttpRequest**

paginile Web nu mai trebuie reîncărcate complet,
conținutul lor – structurat via HTML –
fiind manipulat prin DOM în cadrul *browser*-ului,
în conformitate cu datele recepționate de la server

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

open ( )

inițiază – deschide – o conexiune HTTP cu serverul, emițând o cerere: GET, POST,...

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

send ( )

transmite (asincron) date – *e.g.*, JSON, XML etc. –, spre aplicația/serviciul ce rulează pe server

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

send ( )

transmite (asincron) date – *e.g.*, JSON, XML etc. –, spre aplicația/serviciul ce rulează pe server

orice *listener* (asociat evenimentelor **onloadstart**, **onprogress**, **onload**, **onloadend**, **ontimeout**, **onabort**, **onerror**) trebuie stabilit înainte de a trimite date

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

abort ( )

abandonează transferul de date curent

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

setRequestHeader ( )

specifică anumite câmpuri de antet HTTP

exemple: **Cookie**, **Keep-Alive**, **User-Agent**,...

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

getResponseHeader ( )

furnizează valoarea unui anumit câmp prezent
în antetul mesajului de răspuns HTTP trimis de server

interacțiune web: ajax

Metode importante oferite de **XMLHttpRequest**

getAllResponseHeaders ( )

oferă toate câmpurile HTTP trimise de server, exceptând **Set-Cookie**

interacțiune web: ajax

Proprietăți de bază ale **XMLHttpRequest**

readyState

furnizează codul de stare a transferului:

0 – **UNSENT**

1 – **OPENED**

2 – **HEADERS_RECEIVED**

3 – **LOADING**

4 – **DONE**

interacțiune web: ajax

Proprietăți de bază ale **XMLHttpRequest**

status

oferă codul de stare HTTP întors de serverul Web:
200 (*Ok*)
404 (*Not Found*)
500 (*Internal Server Error*)
...

interacțiune web: ajax

Proprietăți de bază ale **XMLHttpRequest**

statusText

conține mesajul corespunzător codului de stare HTTP

interacțiune web: ajax

Proprietăți de bază ale **XMLHttpRequest**

responseText
responseXML

stochează răspunsul (datele) obținut(e) de la server

interacțiune web: ajax

Proprietăți de bază ale **XMLHttpRequest**

onreadystatechange

specifică funcția ce va fi invocată la modificările de stare ale transferului de date dintre server și client

handler de tratare a evenimentelor de transfer

interacțiune web: ajax

Excepții ce pot fi emise

AbortError
InvalidAccessError
InvalidStateError
NetworkError
SecurityError
TimeoutError
...

Alte facilități:
datele vehiculate pot fi de mai multe tipuri
(**ArrayBuffer**, **Blob**, **Document**, **DOMString**, **FormData**)

detalii la **xhr.spec.whatwg.org/#interface-formdata**

Alte facilități:
procesul de transmitere a datelor spre server (*upload*)
poate avea asociat un *handler* specific
via proprietatea **upload**

Alte facilități:
progresul încărcării poate fi urmărit pe baza funcționalităților specificate de interfața **ProgressEvent**

xhr.spec.whatwg.org/#interface-progressevent

Alte facilități:
stabilirea unui **timeout** privind realizarea unei cereri
(la nivel de milisecunde)

o valoare nenulă cauzează realizarea
unei preîncărcări (*fetching*) a resursei

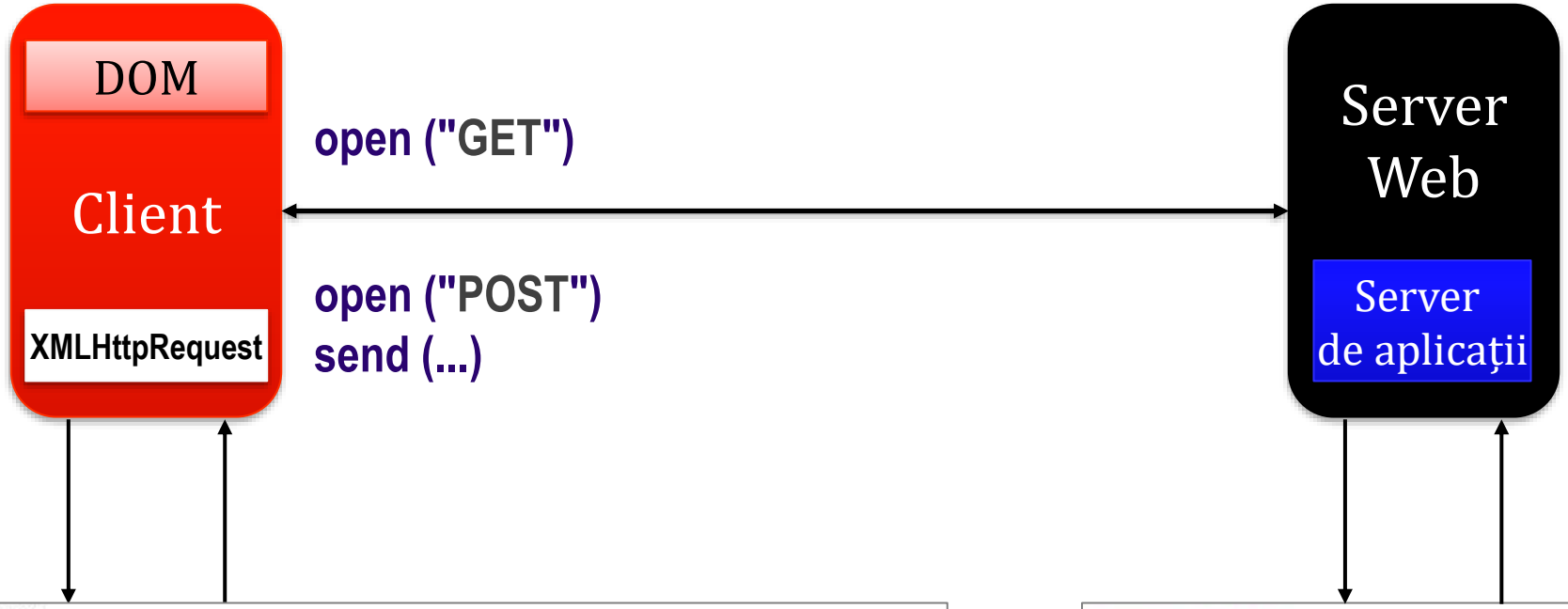
Complementar, a se experimenta **Fetch**
(*HTML Living Standard*, 12 mai 2021)

API oferit de orice navigator Web modern pentru
a specifica accesul la date (reprezentări de resurse)
în urma inițierii unei cereri HTTP și
obținerii unui răspuns furnizat de server

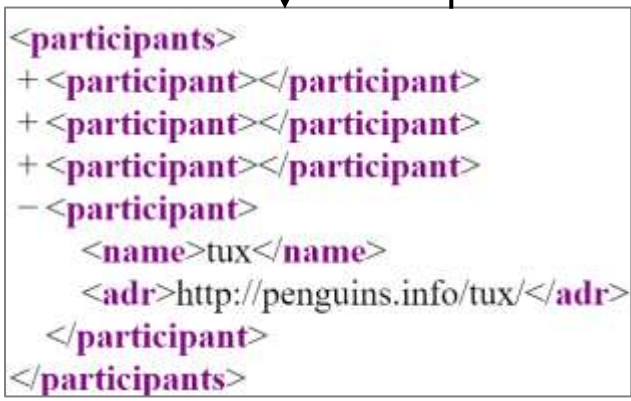
fetch.spec.whatwg.org

interacțiune web: ajax – studiu de caz

Verificarea existenței unui nume de utilizator
în vederea creării unui cont de autentificare
în cadrul unei aplicații Web



fereastra navigatorului



date XML (pe server)

verificarea asincronă a existenței unui cont pe server

interacțiune web: ajax – studiu de caz

Verificarea existenței unui nume de utilizator în vederea creării unui cont de autentificare în cadrul unei aplicații Web

tratând prin DOM evenimentul **onblur**, putem detecta – interogând asincron aplicația Web de pe server – faptul că numele de cont introdus de utilizator într-un formular Web deja a fost folosit de altcineva

interacțiune web: ajax – studiu de caz

Verificarea existenței unui nume de utilizator
în vederea creării unui cont de autentificare
în cadrul unei aplicații Web

aplicația Web de pe server – adoptând stilul REST –
va oferi un document XML modelând răspunsul la
interogarea „există deja un utilizator având un nume dat?”

```php
<?php                    // program PHP, cu rol de serviciu Web, rulat la nivel de server
define ('DOCXML', './particip.xml'); // locația documentului XML
// trimitem tipul conținutului; aici, XML
header ('Content-type: text/xml');

// funcție care verifică dacă un nume de participant deja există
// returnează 1 dacă numele există, 0 în caz contrar
function checkIfNameExists ($aName) {
  // încărcăm datele despre participanți via SimpleXML
  if (!($xml = simplexml_load_file (DOCXML))) { return 0; }
  // parcurgem toți participanții găsiți cu XPath...
  foreach ($xml->xpath('/participants/participant/name') as $name) {
    // comparăm numele, ignorând minusculele de majuscule
    if (!strcasecmp($aName, $name)) { return 1; }
  }
  return 0;
}
?>
<response>
  <result><?php echo checkIfNameExists ($_REQUEST['name']); ?></result>
</response>
```

```html
<!-- Formularul Web preluând date de la utilizator -->
<form action="add.php" method="post">
    <div>
      <label for="name">Account name:</label>
      <input type="text" name="name" id="name"
              placeholder="Enter the desired account name"
              onblur="javascript:signalNameExists (this.value, ")" />
      <!-- mesaj inițial ascuns -->
      <span class="hidden" id="errName">
        Name already exists, choose another one...
      </span>
    </div>

    <div>
      <label for="adr">Address:</label>
      <input type="text" name="adr" id="adr" />
    </div>

    <input type="submit" value="Apply" />
</form>
```

```css
/* stiluri CSS */
.err { /* mesajul de eroare */
  display: inline;
  color: red;
  background-color: yellow;
  font-family: sans-serif;
}
.hidden { /* mesaj ascuns */
  display: none;
}
```

```
// programul JS – varianta clasică – executat în cadrul browser-ului
var request; // încapsulează cererea HTTP către serverul Web

function loadXML (url) { // încarcă un document XML desemnat de 'url'
  // verificăm existența obiectului XMLHttpRequest (browser antic?)
  if (window.XMLHttpRequest) {
    request = new XMLHttpRequest ();  // există suport nativ
  } else
    if (window.ActiveXObject) {  // se poate folosi obiectul ActiveX din MSIE
      request = new ActiveXObject ("Microsoft.XMLHTTP");
    }
  if (request) { // există suport pentru Ajax
    // stabilim funcția de tratare a stării transferului de date
    request.onreadystatechange = handleResponse;
    // preluăm documentul prin metoda GET
    request.open ("GET", url, true);
    request.send (null);  // nu trimitem nimic serviciului Web
  }
}
```

```
// funcția de tratare a schimbării de stare a cererii
function handleResponse () {
  // verificăm dacă încărcarea s-a terminat cu succes
  if (request.readyState == 4) {
    if (request.status == 200) {   // am obținut codul de stare '200 Ok'?
      // procesăm datele recepționate prin DOM
      // (preluăm elementul rădăcină al documentului XML)
      var response = request.responseXML.documentElement;
      var res = response.getElementsByTagName('result')[0].firstChild.data;
        // apelăm o funcție ce va modifica arborele DOM al paginii Web
        // conform răspunsului transmis de serviciul invocat,
        // pentru a semnala (in)existența numelui
        signalNameExists ('', res);
    }
    // eventual, se pot trata și alte coduri HTTP (404, 500 etc.)
  else {
    console.error ("A problem occurred (XML data transfer):\n"
                                                    + request.statusText);
  }
}
```

```
// funcția de semnalare a existenței unui nume de persoană
function signalNameExists (name, exist) {
  // avem un răspuns?
  if (exist != '') {
    // preluăm via metodele DOM elementul cu id='eroareNume'
    // pentru a afișa mesajul de eroare
    var msg = document.getElementById ('errName');
    // schimbăm stilul de afișare (în caz de eroare, vor fi aplicate
    // proprietățile CSS din clasa 'eroare', altfel textul va fi ascuns)
    msg.className = exist == 1 ? 'err' : 'hidden';
  } else {
    // nu e niciun răspuns stabilit, vom verifica asincron
    // trimițând o cerere către server
    loadXML ('verify.php?name=' + name);
  }
}
```

de studiat
exemplul din arhivă

A new participant

Account name:

tux

Name already exists, choose another one...

Address:

Your address (e-mail or URI)

Apply

Inspector | Console | Debugger | Network | DOM

All | HTML | CSS | JS | XHR | Fonts | Images | Media | Flash | WS | Other | 3 requests, 0.13 KB, 28.91 s | Filter URLs

Status	Method	File	Domain	Cause
200	GET	verify.php?name=tux	localhost	xhr
200	GET	verify.php?name=busaco	localhost	xhr
200	GET	verify.php?name=tux	localhost	xhr

Headers | Cookies | Params | Response

Response payload

```
1 <response>
2   <result>1</result>
3 </response>
```

utilizatorul introduce un nume de cont; via Ajax, i se va semnala că deja există, conform răspunsului XML trimis de către serviciul Web

cerere HTTP via URL-ul **http://web.info/verify.php?name=tux**

răspuns XML de forma **<response><result>1</result></response>**

0 = nu există

studiu de caz: RandomAjax

HTML

```html
<div id="numbers">[Wait, please...]</div>
```

CSS

```css
#numbers {
    font-family: monospace;
    font-size: 1em;
    width: 12em;
}
```

JAVASCRIPT

```javascript
// a JS program that asynchronously get a sequence of numbers generated by random.o
const URL = 'https://www.random.org/sequences/?min=1&max=33&col=1&format=plain&rnd=new';
const TIME = 2000;

try { // trying to instantiate a XMLHttpRequest object
    var xhr = new XMLHttpRequest();
} catch (e) {
    numbers.textContent = 'XMLHttpRequest cannot be instantiated: ' + e.message;
} finally {
    var numbers = document.getElementById('numbers');
    xhr.ontimeout = function () { numbers.textContent = 'Time-out... :('; };
    xhr.onload = function () {
        if (xhr.readyState === 4) {   // data arrived
            if (xhr.status === 200) { // response Ok from Web service
                // substituting white spaces with comma and
                // putting the content into the HTML element identified by 'numbers'
                numbers.textContent = xhr.responseText.trim().replace(/\W+/g, ', ');
            } else {
                numbers.textContent = 'An error occurred: ' + xhr.statusText;
            }
        }
    };
    xhr.open("GET", URL, true); // opening connection
    xhr.timeout = TIME;         // setting the response time
    xhr.send(null);             // sending the HTTP request (no data is provided)
}
```

```
5, 18, 13, 32, 6,
21, 17, 24, 14, 30,
20, 15, 16, 4, 8,
33, 31, 29, 10, 19,
7, 27, 28, 2, 11,
12, 1, 22, 9, 25,
26, 23, 3
```

preia asincron
o secvență de numere
aleatoare generate de
random.org – trimisă ca
text obișnuit

jsfiddle.net/busaco/2254kdqn/

```
var xhr = new XMLHttpRequest ();
var numbers = document.getElementById ('numbers’);

// tratarea evenimentului de expirare a timpului de așteptare
xhr.ontimeout = function () { numbers.textContent = 'Time-out... :('; };
// tratarea evenimentului de preluare a datelor solicitate unui serviciu
xhr.onload = function () {
   if (xhr.readyState === 4) { // am primit datele
     if (xhr.status === 200) {   // răspuns Ok din partea serverului
       // înlocuim spațiile albe cu virgulă și plasăm conținutul
       // în cadrul elementului HTML identificat prin 'numbers'
       numbers.textContent = xhr.responseText.trim ().replace (/\W+/g, ', ');
     } else {
       numbers.textContent = 'An error occurred: ' + xhr.statusText;
     }
   }
};
```

```
// adresa Web a sursei de date
const URL =  'https://www.random.org/sequences/
                    ?min=1&max=33&col=1&format=plain';
// timpul maxim de așteptare a răspunsului trimis de server
const TIME = 2000;

// deschidem conexiunea în mod asincron
xhr.open ("GET", URL, true);
// stabilim timpul maxim de așteptare a răspunsului
xhr.timeout = TIME;
// nu expediem date
xhr.send (null);
```

studiu de caz: RandomAjax (Fetch)

HTML

```html
<div id="numbers">[Wait, please...]</div>
```

CSS

```css
#numbers {
  font-family: monospace;
  font-size: 1em;
  width: 12em;
}
```

JAVASCRIPT

```javascript
// A JS program that asynchronously get (by using the Fetch API) a sequence of numbers
by random.org. See also "Introduction to fetch()": https://developers.google.com/web/updates
/2015/03/introduction-to-fetch
const URL = 'https://www.random.org/sequences/?min=1&max=33&col=1&format=plain&rnd=new';

function status(response) { // using promises to handle the returned HTTP status code
  if (response.status >= 200 && response.status < 300) {
    return Promise.resolve(response)
  } else {
    return Promise.reject(new Error(response.statusText))
  }
}

fetch(URL)
  .then(status) // checking if data was successfully received
  .then((response) => response.text()) // transforming received data into a string
  .then(function(response) { // processing the number sequence
    // substituting white spaces with comma and
    // putting the content into the HTML element identified by 'numbers'
    var numbers = document.getElementById('numbers');
    numbers.textContent = response.trim().replace(/\W+/g, ', ');
  })
  .catch(function(error) { // an error occurred :(
    numbers.textContent = 'An error occurred: ' + error;
  });
```

```
28, 25, 7, 33, 23,
22, 14, 21, 5, 1,
18, 2, 4, 13, 12, 6,
20, 17, 19, 24, 16,
31, 11, 29, 10, 3,
8, 26, 30, 27, 15,
32, 9
```

soluție folosind **Fetch API** pentru aceeași problemă

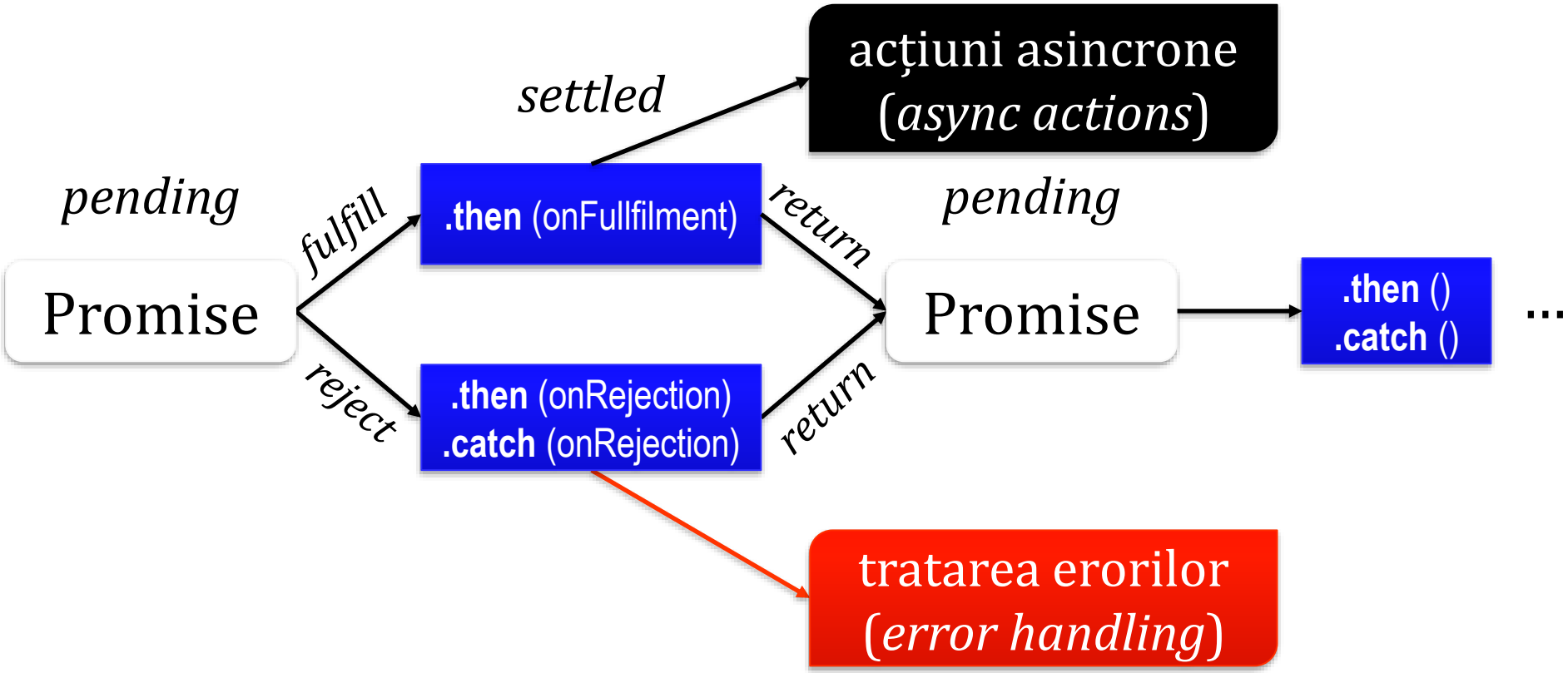
jsfiddle.net/busaco/a2q9regd/

interacțiune web: ajax

Noul API **Fetch** implementat de navigatorul Web
se bazează pe conceptul
promisiune (***promise***) ≡ rezultat ce ar putea fi oferit
în urma execuției unei operații asincrone

represents an operation that has not completed yet,
but is expected in the future

avansat



developer.mozilla.org/Web/JavaScript/Guide/Using_promises
exploringjs.com/es6/ch_promises.html
web.dev/promises/

Recurgem la *promises* pentru a realiza procesări în funcție de codul de stare HTTP primit

```
function status(response) {
   if (response.status >= 200 && response.status < 300) {
     // cererea poate fi rezolvată – fulfill
     return Promise.resolve (response)
   } else {
     // cererea a fost refuzată – reject
     return Promise.reject (new Error (response.statusText))
   }
}
```

```
let numbers = document.getElementById ('numbers');

fetch (URL)
 .then (status) // verificăm dacă datele au fost recepționate cu succes
 // transformăm obiectul răspunsului în șir de caractere
 .then (response => response.text ())
 // procesăm secvența de numere
 .then (response => {
    // înlocuim spațiile albe cu virgulă și plasăm conținutul
    // în cadrul elementului HTML identificat prin 'numbers’
    numbers.textContent = response.trim ().replace (/\W+/g, ', ');
 })
 .catch (error => {                    // a survenit o eroare :(
    numbers.textContent = 'An error occurred: ' + error;
 });
```

developer.mozilla.org/Web/API/Fetch_API

studiu de caz: PostJSON

Scenariu – implementare cu JS modern (ES6):
preluăm caractere de la client – introduse în <textarea> –
și le trimitem asincron prin POST unei aplicații Web
rulând pe server care le expediază înapoi

mesaje vehiculate în format JSON
{ "tasta": "*caracter*", "data": "*secunde*" }

tratăm evenimentul **keypress** pentru a capta tastele
acționate de utilizator

```
// codul programului ES6 interpretat de navigatorul Web
// tratăm evenimentul de apăsare a unei taste
const trateazaEveniment = ev => {
   // mesajul propriu-zis trimis serverului prin POST
   // atunci când survine evenimentul
   let msg = `{ "tasta": "${String.fromCharCode (ev.charCode)}",
                "data": "${Date.now()}" }`;
   // încapsulăm o cerere POST într-un obiect Request
   let request = new Request ('/ajax/post.php', {
     method: 'POST',
     // convertim datele JSON în șir de caractere
     body: JSON.stringify (msg),
     // n-avem câmpuri-antet
     headers: {}
   });
```

de consultat
arhiva cu exemple

```
  fetch (request) // promitem să executăm codul, transmițând cererea...
  .then (response => {    // verificăm dacă am primit date JSON de la server
     let contentType = response.headers.get ('Content-Type');
     if (contentType && contentType.includes('application/json')) {
       return response.json ();
     };
     throw new TypeError ('Datele primite nu-s JSON :(‘);
  })
  .then (json => {    // procesăm efectiv datele
     // creăm via DOM un nod text care indică tasta apăsată
     let elem = document.createTextNode (json.tasta);
     document.getElementById ('tasteApasate').appendChild (elem);
     // raportăm datele primite și la consola browser-ului Web
     console.log
       (`Date JSON primite: tasta=${json.tasta}, data=${Date(json.data)}`);
  })
};
// via DOM, tratăm evenimentul keypress
document.addEventListener ('keypress', trateazaEveniment);
```

```php
// post.php -- program PHP care preia date JSON transmise via POST
// de client și le retrimite acestuia (echo)

function eJSONValid ($sir) {    // verifică dacă datele JSON sunt corecte
   json_decode ($sir);
   return json_last_error () == JSON_ERROR_NONE;
}

// preluăm de la intrarea standard datele transmise de client (raw data)
// (aici, cele dintr-o cerere POST)
$date = trim (file_get_contents ("php://input"));

if (eJSONValid ($date)) { // trimitem datele JSON înapoi dacă sunt în regulă
   header ("Content-type: application/json");
   echo json_decode ($date);
} else
   die ('Date incorecte');
```

programul (serviciul) Web invocat pe server

Redare taste apasate (trimise si unei aplicatii pe server)

Web

Taste: Web

Inspector | Console | Debugger | Performance | Network

Filter URLs | Disable Cache | No Throttling

All | HTML | CSS | JS | XHR | Fonts | Images | Media | WS | Other

St...	Met...	File	Initiator	Transferred	Size
200	POST	post.pl	post-chars.html:26 (fetch)	228 B	41 B
200	POST	post.pl	post-chars.html:26 (fetch)	228 B	41 B
200	POST	post.pl	post-chars.html:26 (fetch)	228 B	41 B

Headers | Cookies | Request | Response

Filter properties

JSON | Text | Raw

1 { "tasta": "W", "data": "1620920514168" }

Inspector | Console | Debugger | Performance | Network

Filter Output | Errors | Warnings | Logs | Info | Debug | CSS | XHR | Requests

XHR POST http://localhost:8080/post.php [HTTP/1.1 200 OK 0ms]

Date JSON primite: tasta=W, data=Thu May 13 2021 18:41:54 GMT+0300 (Eastern European Summer Time) post-chars.html:41:14

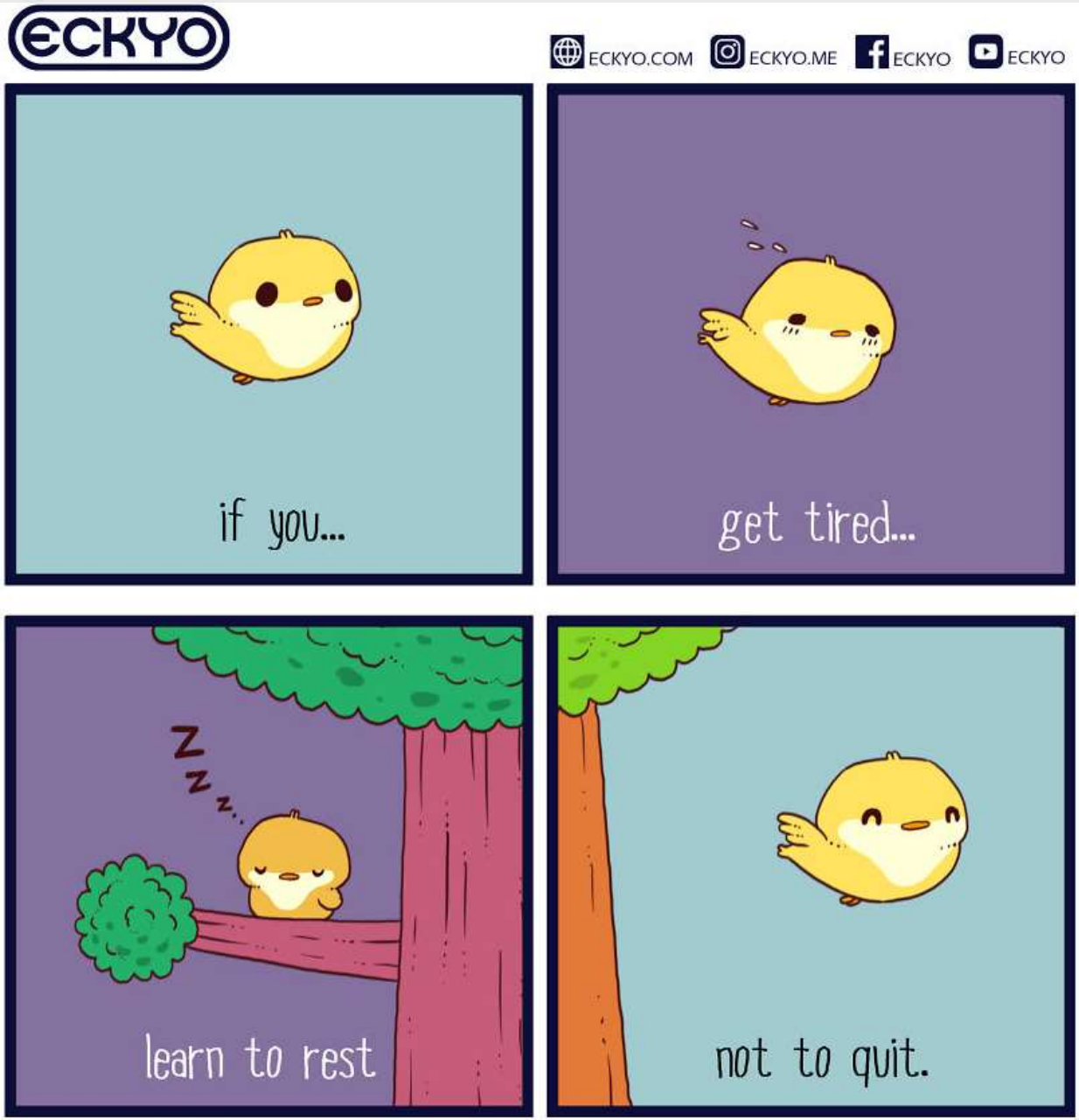
XHR POST http://localhost:8080/post.php [HTTP/1.1 200 OK 0ms]

Date JSON primite: tasta=e, data=Thu May 13 2021 18:41:54 GMT+0300 (Eastern European Summer Time) post-chars.html:41:14

XHR POST http://localhost:8080/post.php [HTTP/1.1 200 OK 0ms]

Date JSON primite: tasta=b, data=Thu May 13 2021 18:41:56 GMT+0300 (Eastern European Summer Time) post-chars.html:41:14

(în loc de) pauză



Ce alte aspecte trebuie considerate atunci când se recurge la Ajax?

interacțiune web: ajax – utilizări

Reîmprospătarea periodică a conținutului

e.g., știri recepționate în formate ca Atom sau RSS, mesaje în cadrul aplicațiilor sociale, notificări,…

interacțiune web: ajax – utilizări

Anticiparea *download*-urilor

pre-încărcarea datelor (*e.g.*, imagini) ce vor fi solicitate

interacțiune web: ajax – utilizări

Auto-completarea datelor

auto-completion
sugestii de căutare – exemplu: Google Suggest

interacțiune web: ajax – utilizări

Validarea în timp-real a datelor introduse
în formulare de către utilizator

exemplificare:
verificarea existenței unui cont sau a unei localități

interacțiune web: ajax – utilizări

Creare de componente de interfață Web (*widgets*)
sau de aplicații Web rulând pe platforme mobile

interacționează cu utilizatorul
pe baza evenimentelor survenite

interacțiune web: ajax – aspecte

Evitarea încărcării întregului document Web

avantaj:
se pot modifica doar fragmente de document

dezavantaj:
bookmarking-ul poate fi compromis
(nu există un URL unic desemnând
reprezentarea resursei curente)

interacțiune web: ajax – aspecte

Oferirea de alternative la Ajax,
atunci când suportul pentru acesta
nu este implementat/activat

graceful degradation

progressive enhancement

www.w3.org/wiki/Graceful_degradation_versus_progressive_enhancement

interacțiune web: ajax – aspecte

Minimizarea traficului dintre *browser* și server

interacțiune web: ajax – aspecte

Transferul de date poate fi monitorizat (+interceptat) via instrumente dedicate

la nivel de *desktop*:
instrumentul WireShark
www.wireshark.org/docs/wsug_html_chunked/

Inspector | Console | Debugger | Network | DOM | Storage

Filter URLs | Persist Logs | Disable cache | No throttling | HAR

All | HTML | CSS | JS | XHR | Fonts | Images | Media | WS | Other

Status	Method	Domain	File	Type	Transferred	Size
200	GET	clients6.google.com	settings?openDrive=true&reaso...	json	6.74 KB	36.98 KB
200	POST	clients6.google.com	batch?$ct=multipart/mixed; bou...	mixed	1.12 KB	374 B
200	GET	drive.google.com	m=oqOEaf,Uxcdvf,syv7,syv8,YPy...	js	127.32 KB	439.68 KB
200	POST	ogs.google.com	get?rt=j&sourceid=49	json	4.90 KB	5.44 KB
200	GET	drive.google.com	m=syyo,sy16u,HLvCjc	js	3.55 KB	7.56 KB
200	GET	clients6.google.com	apps?openDrive=true&reason=...	json	298.53 KB	2.70 MB
200	OPTIONS	clients6.google.com	permissions?openDrive=false&r...	octet-st...	816 B	0 B

134 requests | 33.55 MB / 9.25 MB transferred | Finish: 3.15 min | DOMContentLoaded: 2.22 s | load: 3.15 s

Headers | Cookies | Params | Response | Timings | Stack Trace | Security

Filter properties

JSON

kind: drive#settingList

etag: "Qs5WxPI-aeklfp2kHJBl9C015-c/OFVQmP5N2u-aOW6EQn8clJxc-Es"

selfLink: https://www.
reason=301

items: [...]

0:

kind: drive#setting

etag: "Qs5WxPI-aeklfp2kHJBl9C015-c/A-_FmcbErVTOB3J5-DHWo0cvmDI"

selfLink: https://www.googleapis.com/drive/v2internal/settings/locale?namespace=EXPLORER

inspecție a datelor vehiculate cu instrumentele pentru dezvoltatorii Web oferite de *browser*

interacțiune web: ajax – aspecte

Stabilirea unui mod de interacțiune clar

interacțiune HTML clasică
versus
interacțiune „bogată” cu Ajax
versus
interacțiune la nivelul unei aplicații convenționale

interacțiune web: ajax – aspecte

Adoptarea Ajax pentru creșterea utilizabilității,
nu doar de dragul tehnologiei

exemple negative:
distragerea utilizatorului
abuz de resurse (supradimensionarea arborelui DOM)

interacțiune web: ajax

Ajax oferă premisele invocării asincrone
de (micro-)servicii Web în stilul REST

folosind diverse reprezentări ale datelor transferate:
POX (*Plain Old XML*)
JSON (*JavaScript Object Notation*)
AHAH (*Asynchronous HTML and HTTP*)
text neformatat

interacțiune web: ajax

API-uri specializate

exemplificări:

acces la resurse cartografice – HERE JavaScript APIs

developer.here.com/develop/javascript-api

fotografii de calitate oferite de Unsplash API – Unsplash.js

github.com/unsplash/unsplash-js

Ajax în contextul extensiilor WordPress

codex.wordpress.org/AJAX_in_Plugins

avansat

interacțiune web: ajax

Soluții alternative, moderne:
adoptarea diverselor tehnologii HTML5

server-sent events
WebSocket

pentru detalii, de consultat resursele de la
profs.info.uaic.ro/~busaco/teach/courses/staw/web-film.html#week10

avansat

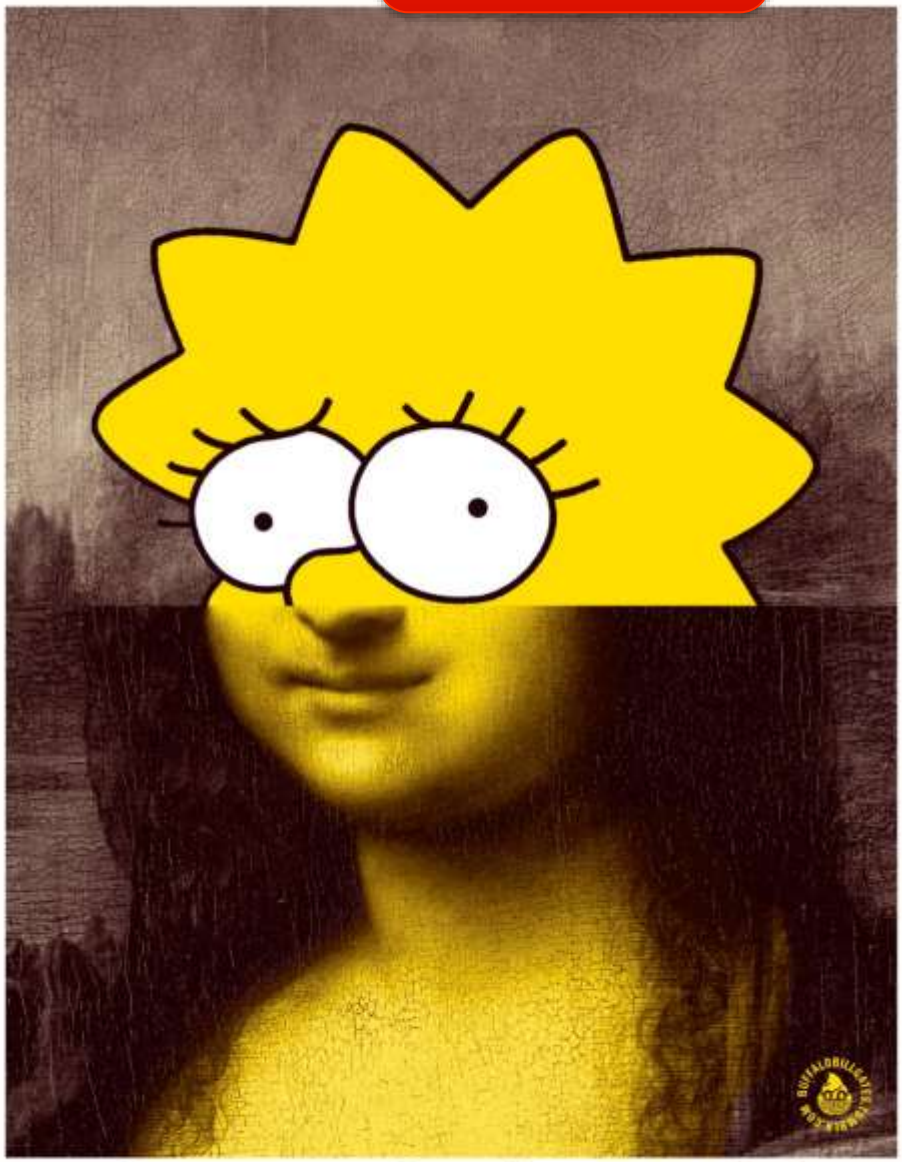
Ajax, alături de existența (micro-)serviciilor și a API-urilor REST/GraphQL, oferă suport pentru dezvoltarea de aplicații de tip:

SPA (*Single Page Application*)

PWA (*Progressive Web Application*)

hibrid (*Web mash-up*)

joc Web – *e.g.*, MMORPG (*Massively Multiplayer Online Role-Playing Game*)



MONA LISA SIMPSON

buffalobillgates.tumblr.com

avansat

interacțiune web: spa

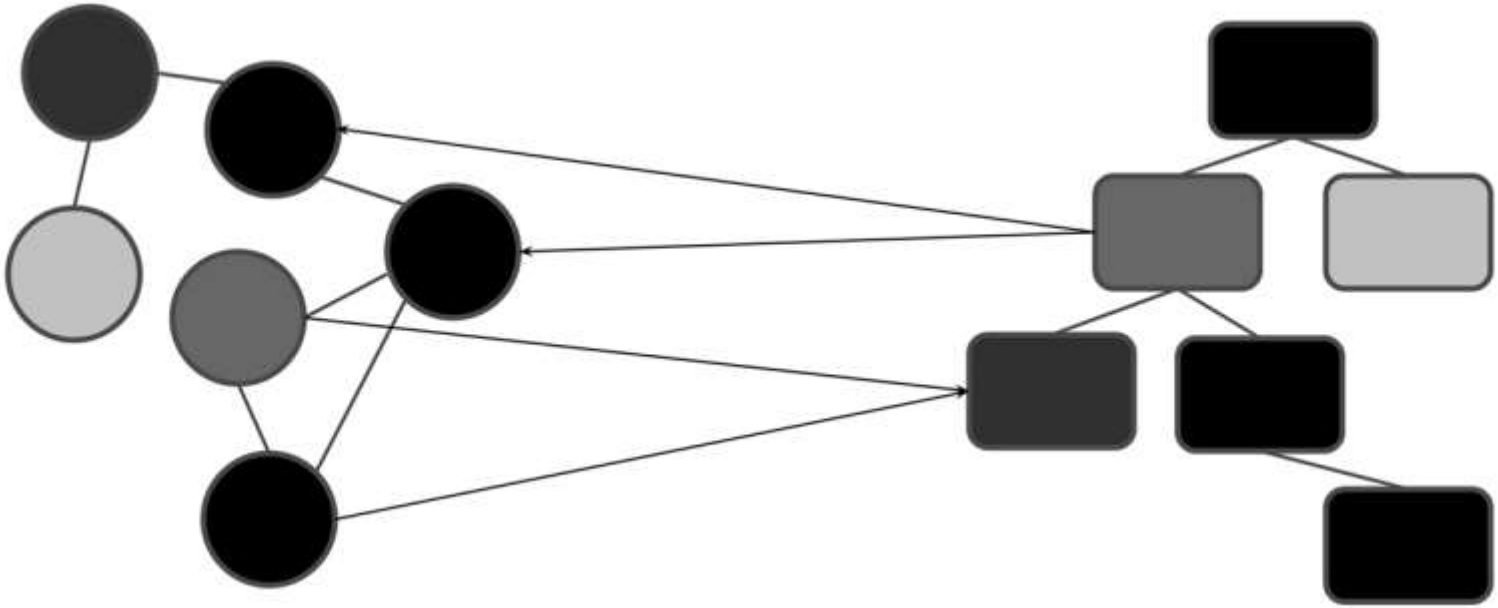
Rescrierea dinamică a conținutului paginii Web – pe baza transferului asincron al datelor – în urma interacțiunii cu utilizatorul

aplicația va fi disponibilă la același URL de bază, recurgând la un unic document HTML, conținutul fiind redat în aceeași fereastră a navigatorului Web

avansat

model de date

componente interactive



la nivel de *front-end*, fiecare componentă interactivă (*e.g.*, buton, *player*, câmp de editare, calendar) poate să se afle într-o anumită stare (*isPressed*, *isPlaying*, *isTyping*,...)

acestă stare nu este neapărat dependentă de *back-end*

avansat

interacțiune web: spa

Problemă:
redarea/împrospătarea eficientă a conținutului
(toate componentele interactive să fie sincronizate)

operațiile de alterare (*mutation*) a DOM-ului
sunt costisitoare pentru aplicații Web complexe
– *e.g.*, o aplicație colaborativă (Draw.io, GitHub, OverLeaf)
sau un joc Web – de pildă, vezi lista
www.mmobomb.com/browsergames/mmorpg

avansat

interacțiune web: spa

Necesitate:
managementul stării globale a aplicației
(*application state*)

avansat

interacțiune web: spa

Soluție:
managementul stării aplicației la momentul rulării
URL ↔ stare

starea (*state*) curentă a *view*-ului e dependentă de URL
uzual, ***/cale/stare*** sau ***#stare***

avansat

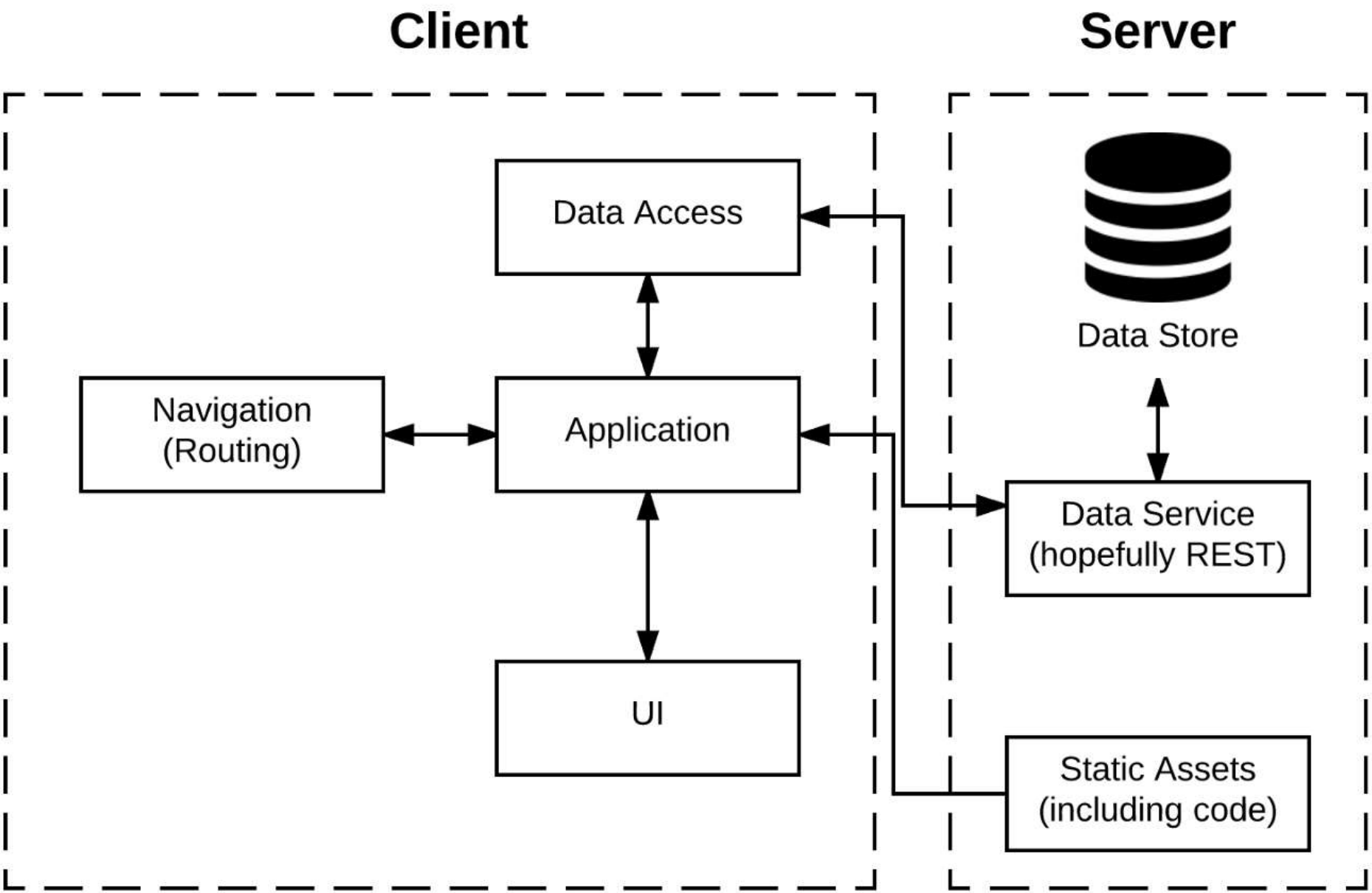
interacțiune web: spa

Soluție:
managementul stării aplicației la momentul rulării
URL ↔ stare

starea (*state*) curentă a *view*-ului e dependentă de URL
uzual, ***/cale/stare*** sau ***#stare***

exemplu real: Google Drive – stări specificate de URL-uri:
/drive/my-drive /drive/recent /drive/shared-with-me /drive/starred

avansat



componente majore ale unei SPA (Patrick Ackerman, 2017)

avansat

La nivel de implementare, managementul stării poate fi facilitat de diverse biblioteci/*framework*-uri JavaScript

Flux (Facebook) – abordare arhitecturală

facebook.github.io/flux/

MobX – *simple, scalable state management*

mobx.js.org

Redux – managementul centralizat al stării aplicației (*a predictable state container*)

a se experimenta și Redux Toolkit și React Redux

redux.js.org

Vuex – managementul stării în aplicații create cu Vue.js (*a state management pattern + library for Vue.js*)

vuex.vuejs.org

avansat

interacțiune web: spa

Uzual, se bazează pe paradigma de
programare (funcțională) reactivă
(FRP – *functional reactive programming*)

vizează fluxuri de date prelucrate asincron
(*event stream*, *observable*)

studii de caz concrete + resurse:
github.com/stoeffel/awesome-fp-js
www.infoq.com/reactive-programming/

avansat

interacțiune web: pwa

Aplicații Web capabile să ofere funcționalități adaptate la contextul de execuție

avansat

interacțiune web: pwa

Aplicații Web capabile să ofere funcționalități adaptate la contextul de execuție

disponibile pe diverse platforme la nivel de navigator Web

recurg la diverse API-uri HTML5 – **platform.html5.org**

developer.mozilla.org/Web/Progressive_web_apps
web.dev/progressive-web-apps/
www.pwastats.com

avansat

interacțiune web: pwa

Caracteristici principale:

adaptare la dispozitiv – *responsive (Web) design*
interacțiuni similare aplicațiilor native
oferă suport pentru *caching*: *offline vs. online*
date transferate „sigur” (cu HTTPS)
dintr-o origine de „încredere”
conținut împrospătat permanent, preluat via servicii Web
instalabile – *e.g.*, eventual, recurgând la un *app store*
partajabile

Putem dezvolta aplicații Web ce combină surse multiple de date/resurse preluate (a)sincron?

mash-ups

Aplicații Web hibride – ***mash-ups***

combinarea – la nivel de client și/sau server –
a conținutului ce provine din surse (situri)
multiple, oferind o funcționalitate/experiență nouă

mash-ups

Exemplificare:

dorim să oferim o aplicație ce pune la dispoziție
informații din domeniul muzical
în funcție de activitățile fizice ale utilizatorului,
pe baza unor servicii Web publice

mash-ups

www.last.fm/api/rest

+

dev.fitbit.com/build/

LaFi
aplicație Web hibridă

API Guides

Introduction
Authentication
Web How-To
Mobile How-To
Desktop How-To
Auth Spec
Scrobbling 2.0 Documentation
Radio API
Playlists API
API Tools
REST Requests
XML-RPC
Terms of Service

API Methods

album
album.addTags
album.getInfo
album.getTags
album.getTopTags
album.removeTag
album.search

API ROOT

The API root URL is located at http://ws.audioscrobbler.com/2.0/

Generally speaking, you will send a method parameter expressed as 'package.method' along with method specific arguments to the root URL. The following parameters are required for all calls:

api_key : A Last.fm API Key.
method : An API method expressed as *package.method*, corresponding to a documented last.fm API method name.

For example:

```
http://ws.audioscrobbler.com/2.0/?method=artist.getSimilar&api_key=xxx...
```

If you are accessing a *write* service, you will need to submit your request as an HTTP POST request. All POST requests should be made to the root url:

```
http://ws.audioscrobbler.com/2.0/
```

With all parameters (including the 'method') sent in the POST body. In order to perform write requests you will need to authenticate a user with the API. See authentication for more.

acces la serviciile Last.fm via o cheie de autentificare

Body and Weight

Full documentation: https://dev.fitbit.com/build/reference/web-api/body/

GET	/1/user/-/body/log/fat/date/{date}.json	Get Body Fat Logs
GET	/1/user/-/body/log/fat/date/{date}/{period}.json	Get Body Fat Logs
GET	/1/user/-/body/log/fat/date/{base-date}/{end-date}.json	Get Body Fat Logs
POST	/1/user/-/body/log/fat.json	Log Body Fat
DELETE	/1/user/-/body/log/fat/{body-fat-log-id}.json	Delete Body Fat Log
GET	/1/user/-/body/log/{goal-type}/goal.json	Get Body Goals
POST	/1/user/-/body/log/fat/goal.json	Update Body Fat Goal
POST	/1/user/-/body/log/weight/goal.json	Update Weight Goal
GET	/1/user/-/body/log/weight/date/{date}.json	Get Weight Logs
GET	/1/user/-/body/log/weight/date/{date}/{period}.json	Get Body Fat Logs

API-ul REST de la FitBit oferă date în format JSON

POST	/1/user/-/body/log/weight.json	Log Weight
DELETE	/1/user/-/body/log/weight/{body-weight-log-id}.json	Delete Weight Log

studiu de caz: DoCa

Crearea unui *mash-up* Web la nivel de client
DoCa (*Dogs 'n' Cats*)

imagini cu câini via **dog.ceo/dog-api/** – răspuns JSON:
{ "status": "success", "message": "*URL_imagine*" }
+
fotografii cu pisici **aws.random.cat/meow** – răspuns JSON:
{ "file": "*URL_imagine*" }

HTML ▼

```html
<div><span id="dog"></span> vs <span id="cat"></span></div>
```

CSS ▼

```css
img {
  width: 200px;
}
```

JavaScript + No-Library (pure JS) ▼

```javascript
// folosim Dog API pentru a obține o imagine aleatorie
// a unui câine dalmațian
fetch(URLDOGS)
  .then(status) // datele au fost recepționate cu succes?
  .then(response => response.json())
  .then(json => {
    // procesăm datele JSON primite...
    // proprietatea 'message' stochează URL-ul imaginii câinelui
    genImg (json.message, 'dog');
  })
  .catch(error => { // redăm eroare survenită
    document.getElementById('dog').textContent = error;
  });

// invocăm random.cat API pentru a obține
// fotografia unei pisici
fetch(URLCATS)
  .then(status) // datele au fost recepționate cu succes?
  .then(response => response.json())
  .then(json => {
    // procesăm datele JSON primite...
    // proprietatea 'file' stochează URL-ul fotografiei cu pisici
    genImg (json.file, 'cat');
  })
  .catch(error => { // redăm eroare survenită
    document.getElementById('cat').textContent = error;
  });
```

vs

utilizăm două apeluri fetch() pentru a prelua date JSON de la cele două servicii Web (API-uri publice)

URL-urile obținute sunt folosite pentru a genera cu DOM elementele <img /> corespunzătoare

codul-sursă complet la **jsfiddle.net/busaco/z2f3vp4m/**

Inspector | Console | Debugger | Network | DOM | Storage

Filter output | Errors | Warnings | Logs | Info | Debug | CSS | XHR | Requests | Persist Log

XHR GET https://dog.ceo/api/breed/dalmatian/images/random [HTTP/2.0 200 OK 128ms]

Headers | Cookies | Params | Response | Timings | Stack Trace | Security

Request URL: https://dog.ceo/api/breed/dalmatian/images/random
Request method: GET
Remote address: 104.27.162.128:443
Status code: 200 OK
Version: HTTP/2.0

Filter headers

Response headers (650 B)

access-control-allow-origin: *
age: 0
cache-control: no-cache, private
cf-cache-status: DYNAMIC
cf-ray: 53203d4b4db9acf4-OTP
content-encoding: br

inspectarea cererilor HTTP inițiate de program

dog.ceo folosește HTTP/2
aws.random.cat recurge la HTTP/1.1

XHR GET https://aws.random.cat/meow [HTTP/1.1 200 OK 128ms]

GET https://images.dog.ceo/breeds/dalmatian/cooper2.jpg [HTTP/2.0 200 OK 78ms]

Headers | Cookies | Params | Response | Timings | Stack Trace | Security

Request URL: https://images.dog.ceo/breeds/dalmatian/cooper2.jpg
Request method: GET
Remote address: 104.27.163.128:443
Status code: 200 OK
Version: HTTP/2.0

Filter headers

Response headers (557 B) Raw headers

```
// verificăm dacă avem succes (codul de stare e 2XX)
function status(response) {
    if (response.status >= 200 && response.status < 300) {
      // cererea poate fi rezolvată
      return Promise.resolve (response)
    } else {
      // cererea a fost rejectată
      return Promise.reject (new Error (response.statusText))
    }
}

// generăm în cadrul unui element identificat prin 'id'
// elementul <img /> pentru a include URL-ul fotografiei
// (parametrul 'url' dat funcției) ce reprezintă un animal
function genImg (url, id) {
    const elem = document.createElement ('img');
    elem.setAttribute ('src', url);
    document.getElementById (id).appendChild (elem);
}
```

```
const URLDOGS = 'https://dog.ceo/api/breed/dalmatian/images/random';

// folosim Dog API pentru a obține o imagine aleatorie
// a unui câine dalmațian
fetch (URLDOGS)
  .then (status)       // datele au fost recepționate cu succes?
  .then (response => response.json ())
  .then (json => {
    // procesăm datele JSON primite...
    // proprietatea 'message' stochează URL-ul imaginii câinelui
    genImg (json.message, 'dog');
  })
  .catch (error => {    // redăm eroarea survenită
    document.getElementById ('dog').textContent = error;
  });
```

```
const URLCATS = 'https://aws.random.cat/meow';

// invocăm random.cat API pentru a obține fotografia unei/unor pisici
fetch (URLCATS)
 .then (status)       // datele au fost recepționate cu succes?
 .then (response => response.json ())
 .then (json => {
  // procesăm datele JSON primite...
  // proprietatea 'file' stochează URL-ul fotografiei cu pisici
  genImg (json.file, 'cat');
 })
 .catch (error => {    // redăm eroarea survenită
  document.getElementById ('cat').textContent = error;
 });
```

avansat

Status	Method	Domain	File	Cause	Type	Transferred	Size
200	GET	dog.ceo	random	fetch	json	704 B	89 B
200	GET	aws.random.cat	meow	fetch	json	430 B	66 B

https://aws.random.cat/meow

Headers Cookies Params Response Timings

Filter properties

JSON

file: https://purr.objects-us-east-1.dream.io/i/nhyM9.jpg

Response payload

```
1 {"file":"https:\/\/purr.objects-us-east-1.dream
```

inspectarea datelor obținute prin transfer asincron cu instrumentele pentru dezvoltatori ale *browser*-ului Web

mash-ups

Se bazează pe fluxuri de ştiri RSS/Atom, servicii Web, API-uri publice,…

„curentul” **SaaS** (*Software As A Service*)

implementare la nivelul:
clientului (*browser*-ului) Web şi/sau serverului Web

mash-ups

Caracteristici:

combinare

vizualizare

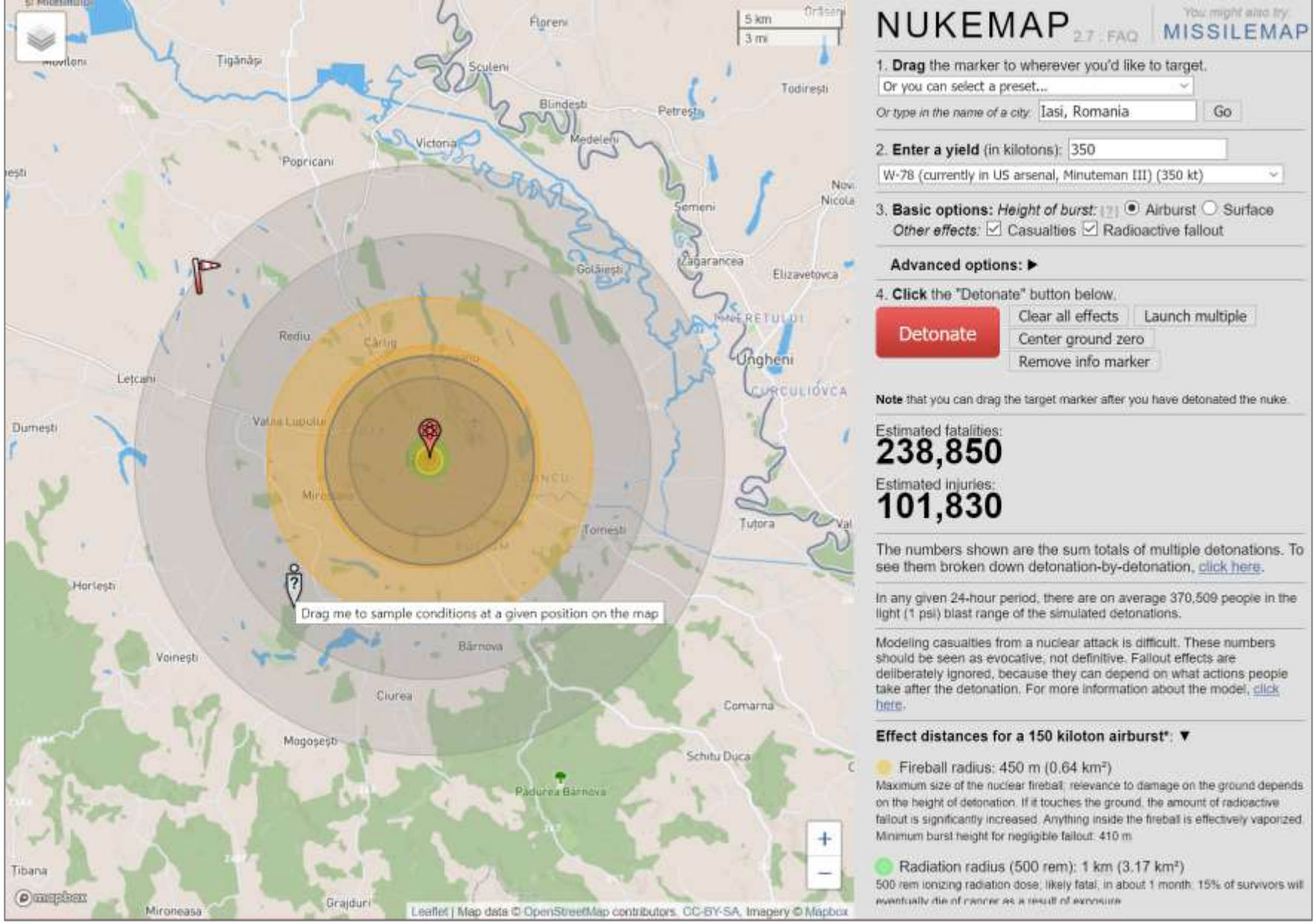
agregare

mash-ups

Combinare

utilizarea de surse de date multiple
poate avea caracter multidimensional

exemplificare:
subiect de interes + locație geografică + moment de timp

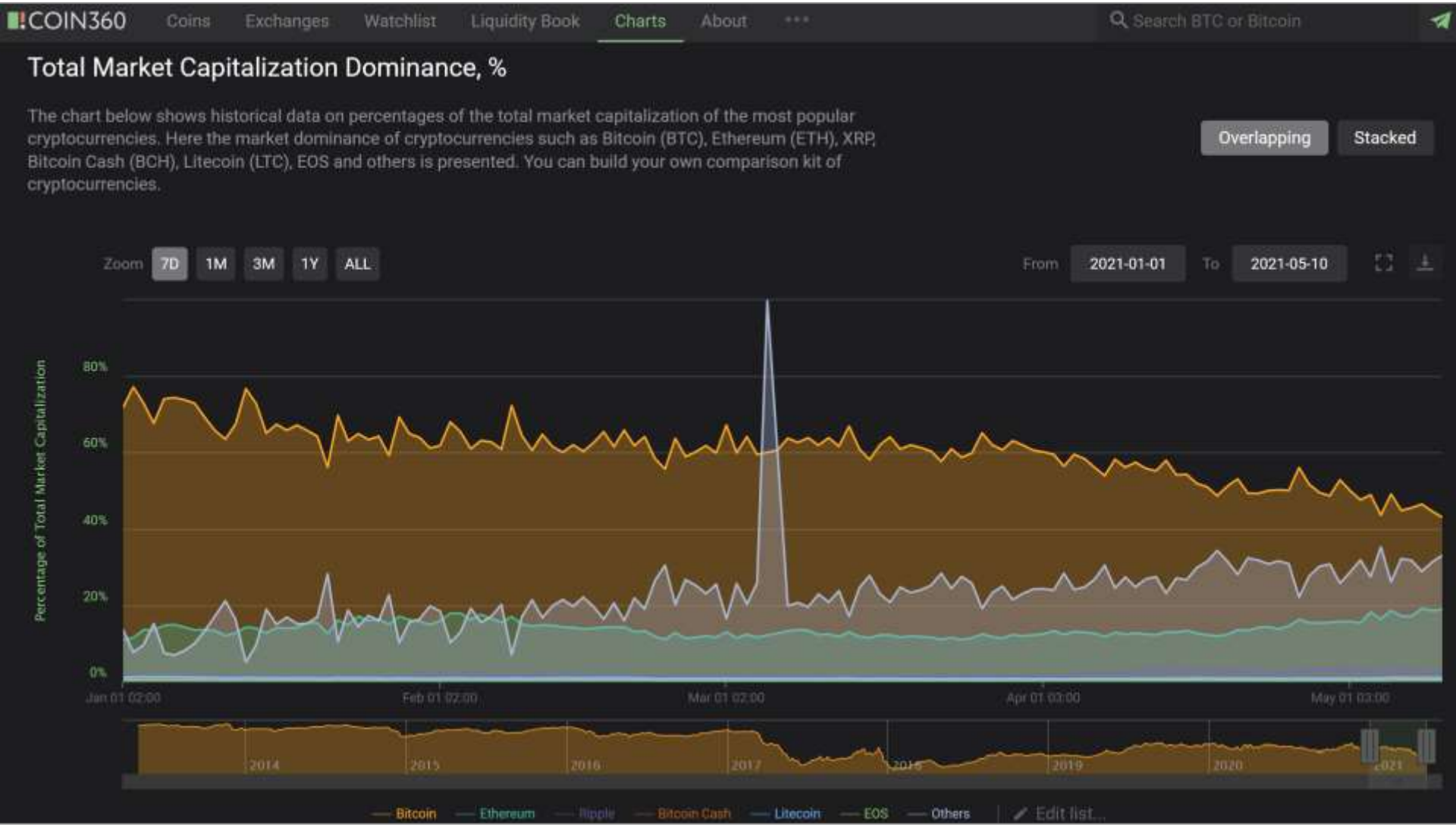


un *mash-up* Web de studiere a efectelor detonării bombelor nucleare – **nuclearsecrecy.com/nukemap/**

mash-ups

Vizualizare

pot fi adoptate diverse tehnici de vizualizare (prezentare) a datelor:
chart-uri, cartografică, *tag cloud*-uri, tridimensională,...



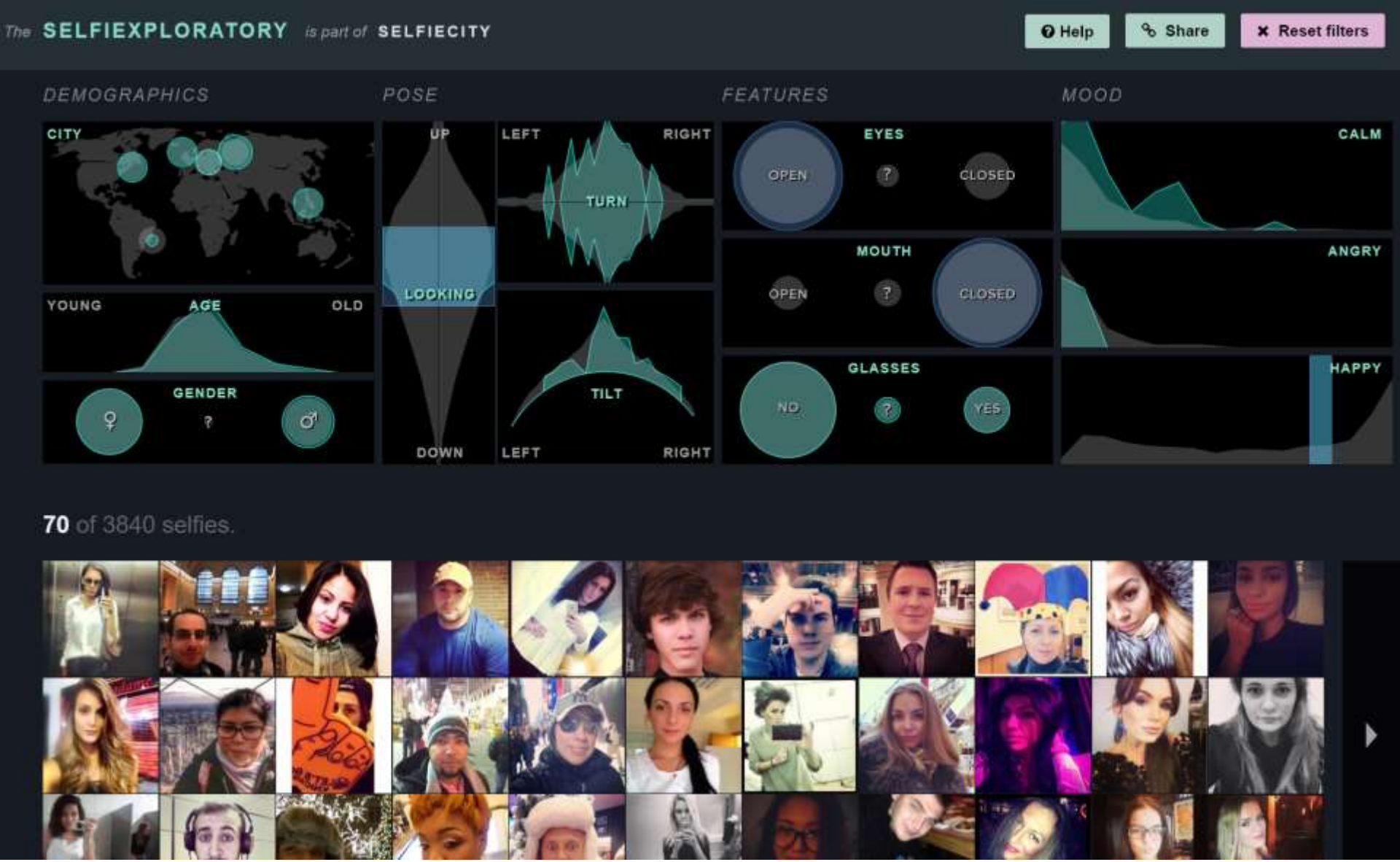
vizualizare în timp-real a evoluției cursului monedelor virtuale pe baza unor surse de date multiple – **coin360.com**

mash-ups

Agregare

gruparea datelor provenite din mai multe surse și analizarea lor: statistici, clasificări, predicții,…

e.g., folosind *machine/deep learning* se pot releva aspecte „ascunse” ale datelor procesate



clasificarea imaginilor expuse de utilizatori
în funcție de vârstă, postură, stare de spirit și altele
Selfiexploratory – **selfiecity.net/selfiexploratory/**

avansat

mash-ups

Surse de date (*data feeds*)	Atom, RSS, geoRSS, micro-date HTML5, RDFa,…
Interfețe de programare (API-uri)	specifice serviciilor publice și vizând procesarea datelor
Biblioteci/*framework*-uri pentru dezvoltare	*framework*-uri Web generice sau oferite de organizații
Instrumente interactive (*Web tools*)	eventual, disponibile în *cloud*
Platforme (*Platform As A Service*)	Digital Ocean, Heroku, Google Cloud Platform, MS Azure,…

LEARN ABOUT APIS | API DIRECTORY | CORONAVIRUS

avansat

Filter Mashups

Data-as-a-Service ✕
API Education ✕

By APIs

Include Deprecated

Mashup Name	Description	Category	Submitted
GroupDocs.Total Free Apps for Windows	GroupDocs.Total Free Apps for Windows is a set of free Windows apps to convert, view & merge...	Applications	04.22.2021
Salonist	Salonist software is a smart tool for handling salon services and activities. With the help of this...	Software-as-a-Service	04.01.2021
WaifuAI Desktop	Talk to cute waifu on Desktop (Windows/Mac /Linux). Waifus are anime-themed personal assistants.	Artificial Intelligence	03.30.2021
WaifuAI Android	Talk to cute waifu on Android. Waifus are anime-themed personal assistants.	Artificial Intelligence	03.30.2021
SimplyCRMApps	Gain more customers, optimize your business and earn more money - with Simply CRM. Simply is the...	B2B	01.13.2021

FEATURED LATEST

FOR API PROVIDERS

What Are APIs and How Do They Work?

Guide to GraphQL: Understanding, Building and Using GraphQL APIs

How Facebook Makes it Nearly Impossible For You To Quit

More for API Providers ›

FOR DEVELOPERS

How to Get Started With Google Actions

How to Build a Monitoring Application With the Google Cloud Vision API

How to Access Any RESTful API Using the R Language

More for Developers ›

ADDITIONAL RESOURCES

API Growth Charts, Industry

lista *mash-up*-urilor: **www.programmableweb.com/mashups/directory**

multe alte API-uri publice la **github.com/toddmotto/public-apis**

Probleme de securitate
privind accesul la resurse via JavaScript?

avansat

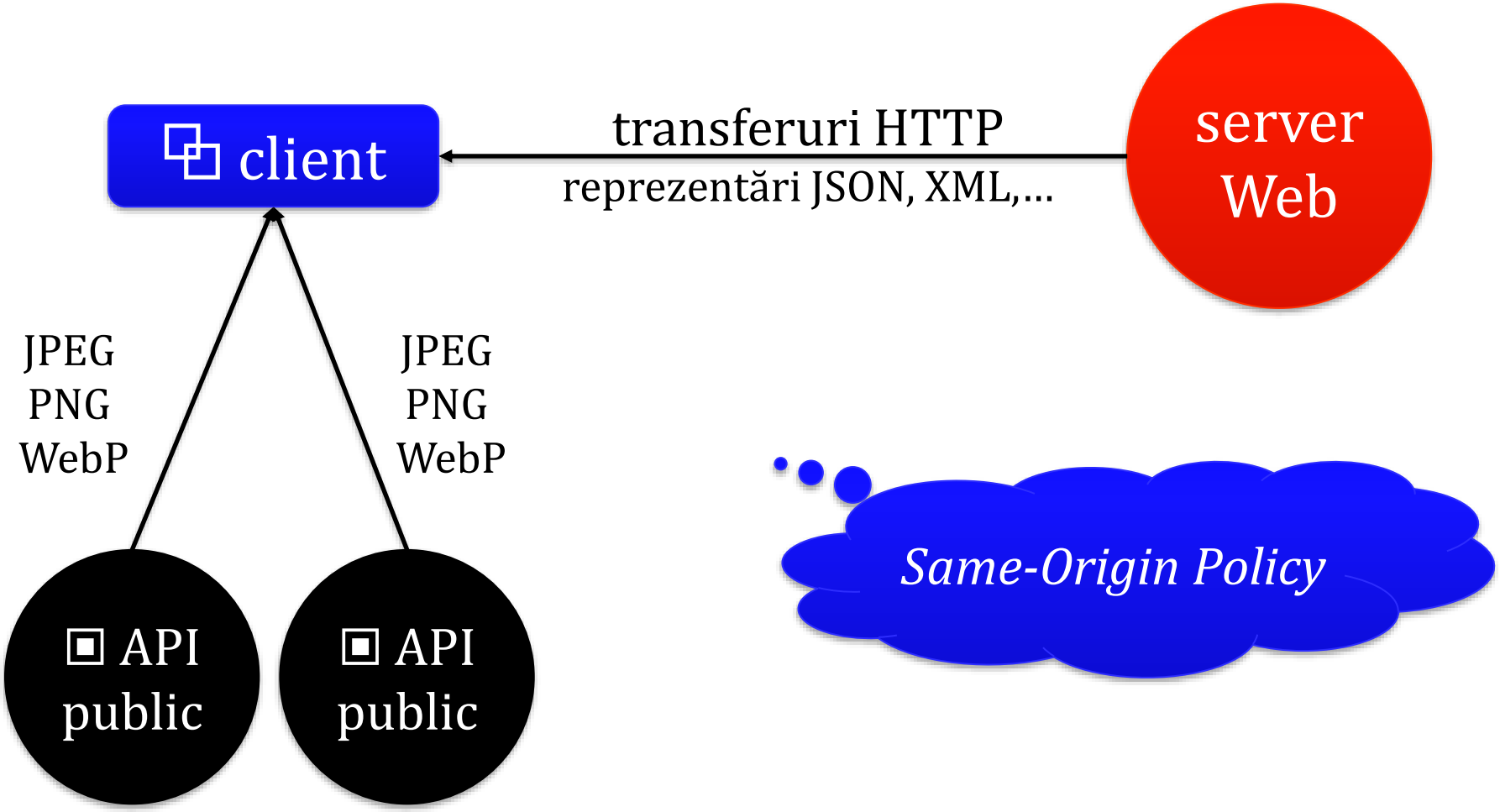
cors

Same-Origin Security Policy

"*restricts how a document or script loaded from one origin can interact with a resource from another origin*"

astfel, un program JavaScript trebuie să acceseze doar datele aparținând aceleași origini – *i.e.*, provenite din același domeniu Internet

avansat



se permit doar transferuri vizând reprezentări de resurse referitoare la imagini (image/*), fișiere CSS și alte programe JavaScript aparținând aceleași origini

avansat

cors

Same-Origin Security Policy

previne cazurile în care un document/program încărcat dintr-o origine să poată accesa/modifica proprietăți ale unui document aparținând altei origini

developer.mozilla.org/Web/Security/Same-origin_policy

avansat

```
const URL = "https://profs.info.uaic.ro/~busaco/teach/courses/web/";

// realizăm o cerere HEAD pentru a obține meta-date despre o resursă
let client = new XMLHttpRequest ();
client.open ("HEAD", URL, true);
client.send ();
client.onreadystatechange = function () {
  // am recepționat câmpurile-antet?
  if (client.readyState == 2) {
    // semnalăm tipul MIME și data ultimei actualizări
    alert ("Resursa de tip '" +
      client.getResponseHeader ("Content-Type") + "' s-a actualizat la " +
      client.getResponseHeader ("Last-Modified"));
  }
}
```

preluarea cu **HEAD** a unor meta-date, în mod asincron

avansat

încercând execuția programului JS în *browser*-ele Web instalate pe calculatorul local (127.0.0.1), se obține:

Inspector | Console | Debugger | Network | DOM | Storage

Filter output | Errors | Warnings | Logs | Info | Debug | CSS | XHR | Requests | Persist Logs

Cross-Origin Request Blocked: The Same Origin Policy disallows reading the remote resource at *https://profs.info.uaic.ro/~busaco/teach/courses/web/*. (Reason: CORS header 'Access-Control-Allow-Origin' missing). [Learn More]

https://developer.mozilla.org/docs/Web/HTTP/CORS/Errors/CORSMissingAllowOrigin

Elements | Console | Sources | Network | Performance | Memory | 1

top | -url:https://profs.info.uai | Default levels

Access to XMLHttpRequest at 'https://profs.info.uaic.ro/~busaco/teach/courses/web/' from origin 'null' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. ajax-head.html:1

URL al altui domeniu ▸ se încalcă *Same Origin Policy*

avansat

caz real: Google Mail

Inspector | Console | Debugger | Network | DOM | Storage | Accessibility

Content Security | 5 hidden | Errors | Warnings | Logs | Info | Debug | CSS | XHR | Requests | Persist

```
Content Security Policy: Ignoring “https:” within script-src: ‘strict-dynamic’ specified
Content Security Policy: Ignoring “http:” within script-src: ‘strict-dynamic’ specified
Content Security Policy: Ignoring ‘x-frame-options’ because of ‘frame-ancestors’ directive.
Content Security Policy: Ignoring “'unsafe-inline'” within script-src or style-src: nonce-source or hash-source specified
Content Security Policy: The page’s settings blocked the loading of a resource at inline (“script-src”).   4   utils.js:35:
4 repeats
Content Security Policy: Ignoring “'unsafe-inline'” within script-src: ‘strict-dynamic’ specified
```

avansat

cors

CORS (***Cross-Origin Resource Sharing***)

permite partajarea la nivel de client a resurselor provenind din origini (*e.g.*, domenii Internet) diferite

astfel, se pot emite cereri între origini (*cross-origin*)

fetch.spec.whatwg.org/#http-cors-protocol

avansat

cors

Politicile de partajare a resurselor se bazează pe anumite câmpuri din antetul mesajelor HTTP

exemplu (se permit doar cereri GET având în antetul mesajului câmpuri specifice din partea unui client aparținând unei origini date):

Access-Control-Request-Method: **GET**
Access-Control-Request-Headers: **Content-Type, Accept**
Origin: **http://localhost:8000**

avansat

cors

Politicile de partajare a resurselor se bazează pe anumite câmpuri din antetul mesajelor HTTP

resurse de interes:
developer.mozilla.org/en-US/docs/Web/HTTP/CORS

de studiat și tutorialul lui Steve Hobbs (2019):
auth0.com/blog/cors-tutorial-a-guide-to-cross-origin-resource-sharing/

avansat

cors: caz real #1

API-ul furnizând imagini publice cu pisici
https://aws.random.cat/meow
acceptă cereri GET emise de orice origine a clientului

Access-Control-Request-Method: GET
Access-Control-Allow-Origin: *

avansat

cors: caz real #2

Google Play acceptă cereri POST emise numai de serviciul Google Drive (originea este https://drive.google.com) ce includ câmpuri-antet vizând autorizarea

Access-Control-Request-Method: **POST**
Origin: **https://drive.google.com**
Access-Control-Request-Headers: **authorization,x-goog-authuser**

câmp nestandardizat

avansat

cors: caz real #3

Reddit.com permite cereri POST și OPTIONS formulate de orice origine, fiind acceptate anumite câmpuri-antet (controlul accesului depinde de timp)

Access-Control-Allow-Headers: **Content-Type**,**Origin**,**Accept**,**X-Signature**
Access-Control-Allow-Methods: **POST**, **OPTIONS**
Access-Control-Allow-Origin: *
Access-Control-Max-age: **1728000**

avansat

JAMstack

Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

"A modern Web development architecture based on client-side JavaScript*, reusable* APIs*, and prebuilt* Markup*."*
Mathias Biilmann

viziune:
jamstack.org

avansat

JAMstack

Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

JavaScript

procesările efective sunt realizate de client (uzual, un *browser* Web) pe baza facilităților oferite de limbajul de programare JS (ECMAScript) și ecosistemului aferent

avansat

JAMstack

Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

APIs

funcționalitățile și datele sunt încapsulate de API-uri reutilizabile – proprii sau oferite de terțe entități – accesate (a)sincron prin HTTPS via programe JS

de revăzut cursurile anterioare:
REST, *serverless*, GraphQL

avansat

JAMstack

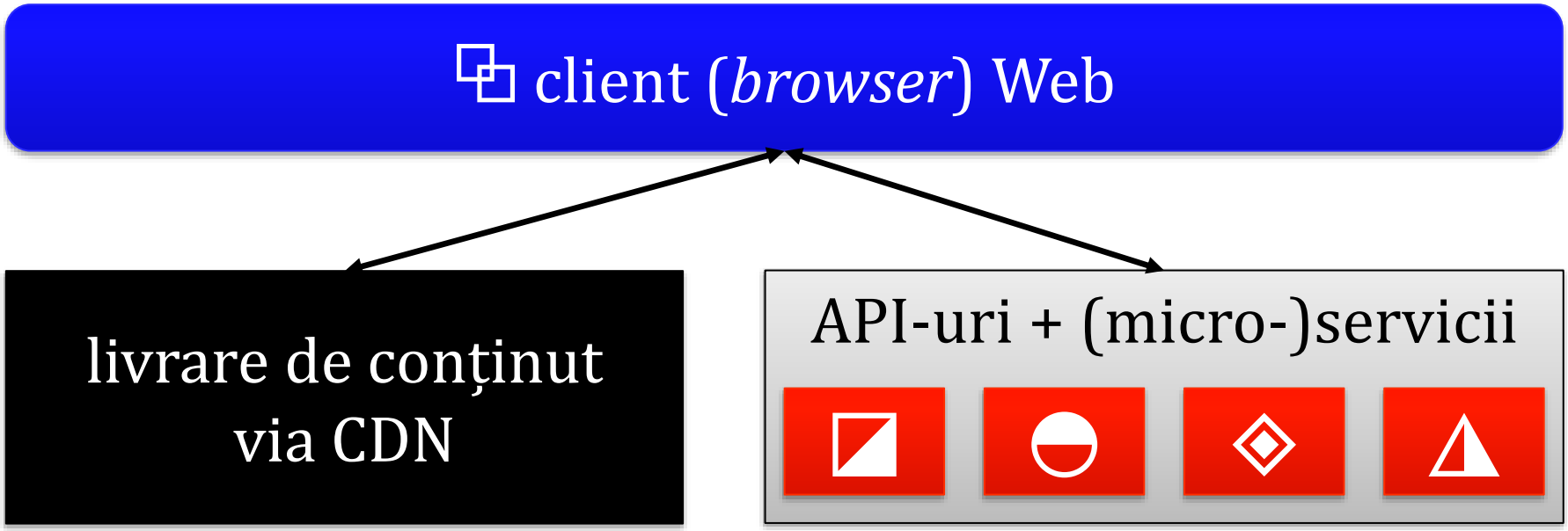
Manieră modernă privind dezvoltarea de aplicații Web la nivel de client

Mark-up

livrarea conținutului via machete de vizualizare (*templates*) definite de limbaje de marcare, pre-procesate în vederea prezentării la client (*pre-rendering*) și stocate redundant de CDN (*Content Distribution Network*), uzual pe baza unor instrumente specifice de tip SSG (*Static Site Generator*) ori *headless* CMS

avansat

JAMstack



resurse + instrumente:

The New Dynamic – **www.tnd.dev**

github.com/automata/awesome-jamstack

avansat

JAMstack: resurse

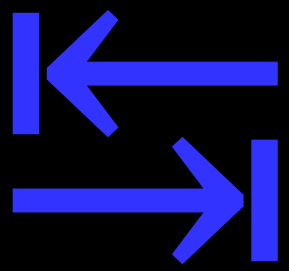
Mathias Biilmann & Phil Hawksworth,
Modern Web Development on the Jamstack, O'Reilly, 2019
www.netlify.com/oreilly-jamstack/

JAMstacked – **jamstack.email**

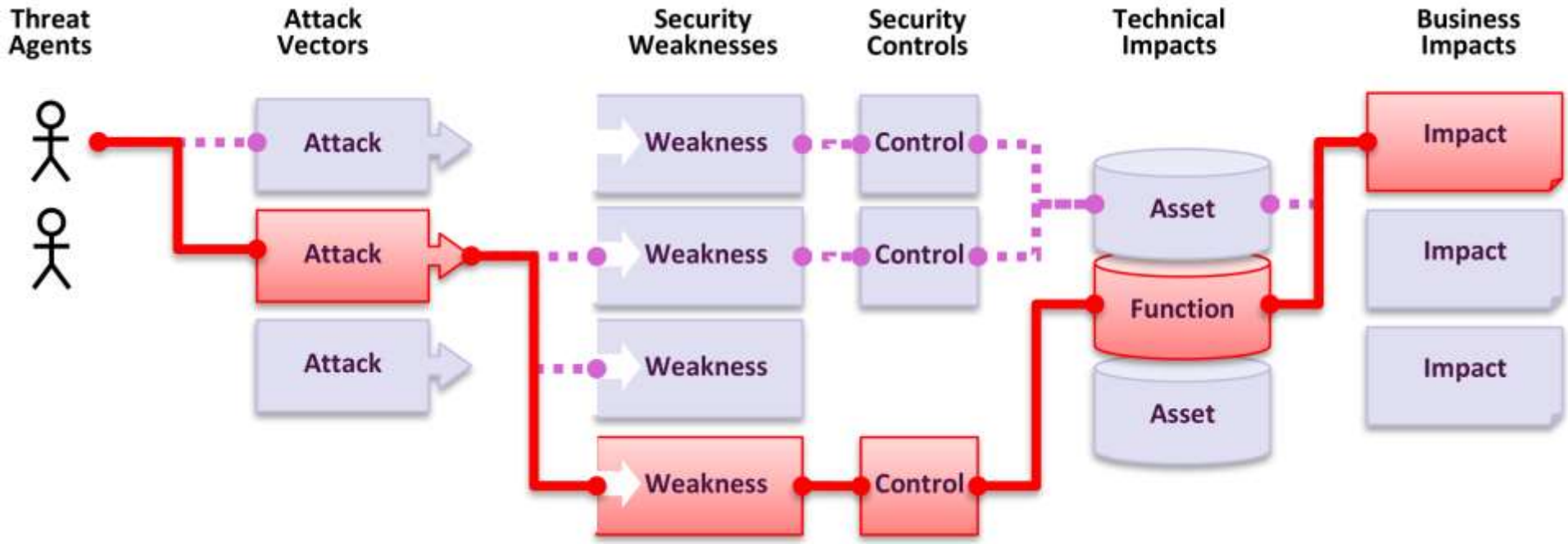
JAMstack podcasts – **thatsmyjamstack.com**

rezumat

interacțiune Web



de la interacțiuni asincrone via Ajax
la *mash-up*-uri Web, CORS și JAMstack



„ultimul” episod: **securitatea aplicațiilor Web**